

Stacks

Stacks

The concept of **procedures** (methods or functions) is a universal feature in programming languages.

These procedures have *local variables* that you can't access outside the procedure.

So the question is:

"Where should these variables be kept in memory?"

The easiest solution is giving each variable a *fixed, absolute memory address*

This won't work, because of **recursion**

a procedure can call itself, and each active call needs its *own* independent copy of its local variables. A single fixed address per variable could only ever hold one call's worth of data at a time.

Why absolute addresses fail

Imagine `factorial(n)` calls `factorial(n-1)`, which calls `factorial(n-2)`, and so on.

If the local variable `n` always lived at the same *absolute* address,

Then every nested call would overwrite the `n` belonging to the call above it.

By the time the recursion unwinds back up, all the outer calls would see the wrong (overwritten) value of `n`.

Each *invocation* of a procedure needs its own private storage for its local variables, and that storage must be created when the call begins and destroyed when the call returns.

Table example

Call	Action	Address of <code>n</code>	Value written	Value <code>factorial</code> <i>should</i> see
<code>factorial(3)</code>	called	<code>200</code>	<code>3</code>	<code>3</code>
<code>factorial(2)</code>	called	<code>200</code>	<code>2</code> (overwrites <code>3</code>)	<code>2</code>
<code>factorial(1)</code>	called	<code>200</code>	<code>1</code> (overwrites <code>2</code>)	<code>1</code>
<code>factorial(1)</code>	returns	<code>200</code>	still <code>1</code>	<code>1</code> (correct, by luck)
<code>factorial(2)</code>	resumes, reads <code>n</code>	<code>200</code>	<code>1</code>	should be <code>2</code> , but reads <code>1</code>
<code>factorial(3)</code>	resumes, reads <code>n</code>	<code>200</code>	<code>1</code>	should be <code>3</code> , but reads <code>1</code>

Every nested call overwrites the same address, so once the recursion starts unwinding, the outer calls can no longer recover their own value of `n`

Stacks

So instead, an area of memory called the **stack** is used to store local variables.

It's reserved for variables, but individual variables do **not** get absolute addresses in it.

Instead, they are accessed through registers like the **Stack Pointer (SP)**.

This means that when a procedure calls itself (or is called by another procedure) or returns, the new local variables get stored in different locations on the stack, and you access them **relative** to the **SP**, rather than through a fixed absolute address.

Stacks

Say we have a program like:

```
1  def func_a():  
2      ...  
3      func_b()  
4      func_d()  
5  
6  def func_b():  
7      ...  
8      func_c()
```

```
1  SP ->  a3 108 |  
2          a2 104 |  
3  LV ->  a1 100 |
```

Stacks

Say we have a program like:

```
1  def func_a():
2      ...
3      func_b()
4      func_d()
5
6  def func_b():
7      ...
8      func_c()
```

```
1  SP ->  b4 124 |
2          b3 120 |
3          b2 116 |
4  LV ->  b1 112 | # remember this address
5          a3 108
6          a2 104
7          a1 100
```

Stacks

Say we have a program like:

```
1  def func_a():  
2      ...  
3      func_b()  
4      func_d()  
5  
6  def func_b():  
7      ...  
8      func_c()
```

```
1  SP ->  c2 132 |  
2  LV ->  c1 128 |  
3          b4 124  
4          b3 120  
5          b2 116  
6          b1 112  
7          a3 108  
8          a2 104  
9          a1 100
```

Stacks

Say we have a program like:

```
1  def func_a():  
2      ...  
3      func_b()  
4      func_d()  
5  
6  def func_b():  
7      ...  
8      func_c()
```

```
1  SP ->  b4 124 |  
2          b3 120 |  
3          b2 116 |  
4  LV ->  b1 112 |  
5          a3 108 |  
6          a2 104 |  
7          a1 100 |
```

Stacks

Say we have a program like:

```
1  def func_a():  
2      ...  
3      func_b()  
4      func_d()  
5  
6  def func_b():  
7      ...  
8      func_c()
```

```
1  SP ->  a3 108 |  
2          a2 104 |  
3  LV ->  a1 100 |
```

Stacks

Say we have a program like:

```
1  def func_a():  
2      ...  
3      func_b()  
4      func_d()  
5  
6  def func_b():  
7      ...  
8      func_c()
```

```
1  SP ->  d5 128 |  
2          d4 124 |  
3          d3 120 |  
4          d2 116 |  
5  LV ->  d1 112 |  
6          a3 108 |  
7          a2 104 |  
8          a1 100 |
```

Operand stacks

Some, but not all, computers use another stack called the **operand stack** to hold temporary values for calculations.

Imagine the code:

```
1  a1 = a2 + a3;
```

One way of doing this is to push the operands onto the stack, then execute the calculation, and finally pop the result off the stack into the variable `a1`.

```
1  SP ->  a3
2           a2
3  LV ->  a1
```

An interesting thing about the operand stack is that it can be **mixed** with the local variable stack, so data in the stack can be used in calculations with data on the operand stack.

Worth noting again that not all systems use an operand stack, but the JVM and IJVM do.

Operand stacks

Some, but not all, computers use another stack called the **operand stack** to hold temporary values for calculations.

Imagine the code:

```
1  a1 = a2 + a3;
```

One way of doing this is to push the operands onto the stack, then execute the calculation, and finally pop the result off the stack into the variable `a1`.

```
1  SP -> |a2|  
2          a3  
3          a2  
4  LV -> a1
```

An interesting thing about the operand stack is that it can be **mixed** with the local variable stack, so data in the stack can be used in calculations with data on the operand stack.

Worth noting again that not all systems use an operand stack, but the JVM and IJVM do.

Operand stacks

Some, but not all, computers use another stack called the **operand stack** to hold temporary values for calculations.

Imagine the code:

```
1  a1 = a2 + a3;
```

One way of doing this is to push the operands onto the stack, then execute the calculation, and finally pop the result off the stack into the variable `a1`.

```
1  SP -> |a3|
2      |a2|
3      a3
4      a2
5  LV ->  a1
```

An interesting thing about the operand stack is that it can be **mixed** with the local variable stack, so data in the stack can be used in calculations with data on the operand stack.

Worth noting again that not all systems use an operand stack, but the JVM and IJVM do.

Operand stacks

Some, but not all, computers use another stack called the **operand stack** to hold temporary values for calculations.

Imagine the code:

```
1  a1 = a2 + a3;
```

One way of doing this is to push the operands onto the stack, then execute the calculation, and finally pop the result off the stack into the variable `a1`.

```
1  SP -> |a2+a3|  
2          a3  
3          a2  
4  LV ->  a1
```

An interesting thing about the operand stack is that it can be **mixed** with the local variable stack, so data in the stack can be used in calculations with data on the operand stack.

Worth noting again that not all systems use an operand stack, but the JVM and IJVM do.

Operand stacks

Some, but not all, computers use another stack called the **operand stack** to hold temporary values for calculations.

Imagine the code:

```
1  a1 = a2 + a3;
```

One way of doing this is to push the operands onto the stack, then execute the calculation, and finally pop the result off the stack into the variable `a1`.

```
1  SP ->  a3
2      a2
3  LV ->  a2+a3
```

An interesting thing about the operand stack is that it can be **mixed** with the local variable stack, so data in the stack can be used in calculations with data on the operand stack.

Worth noting again that not all systems use an operand stack, but the JVM and IJVM do.

Why bother with an operand stack?

An operand stack avoids the need for extra "scratch" registers to hold intermediate results during an arithmetic computation.

Computing $a1 = (a2 + a3) * a4$ without an operand stack would require somewhere to temporarily hold $a2 + a3$ before multiplying by $a4$. With an operand stack, that intermediate value is simply the top of the stack after the `ADD`, ready to be consumed by the next `MUL`.

This design is common in **stack machines** (of which the JVM/IJVM is one)

Instead of instructions naming source and destination registers explicitly (like `ADD R1, R2, R3` in a register machine),

instructions like `IADD` implicitly operate on whatever is currently on top of the operand stack. This tends to make instruction encodings smaller, at the cost of needing more instructions to push/pop values.

The JVM Memory Model

The JVM Memory Model

The JVM has an array of `4,294,967,296` bytes of memory (2^{32}), or

equivalently an array of `1,073,741,824` words (2^{30}), since each word is 4 bytes.

And unlike most ISAs, the JVM makes *no absolute memory addresses* directly visible to the programmer, the only way to access memory is through registers and **increments** of those registers.

These registers are:

1. **the constant pool** (`CPP`): it can't be written to, and contains constants used by the program
2. **the local variable pointer** (`LV`): points to the base of the current procedure's local variables in the stack
3. **the operand stack pointer** (`SP`): points to the top of the operand stack, directly above the `LV` frame
4. **the method area** (`PC`): holds the program itself, and contains the address of the *next instruction*

Why hide absolute addresses?

Hiding absolute addresses is a deliberate portability and safety decision.

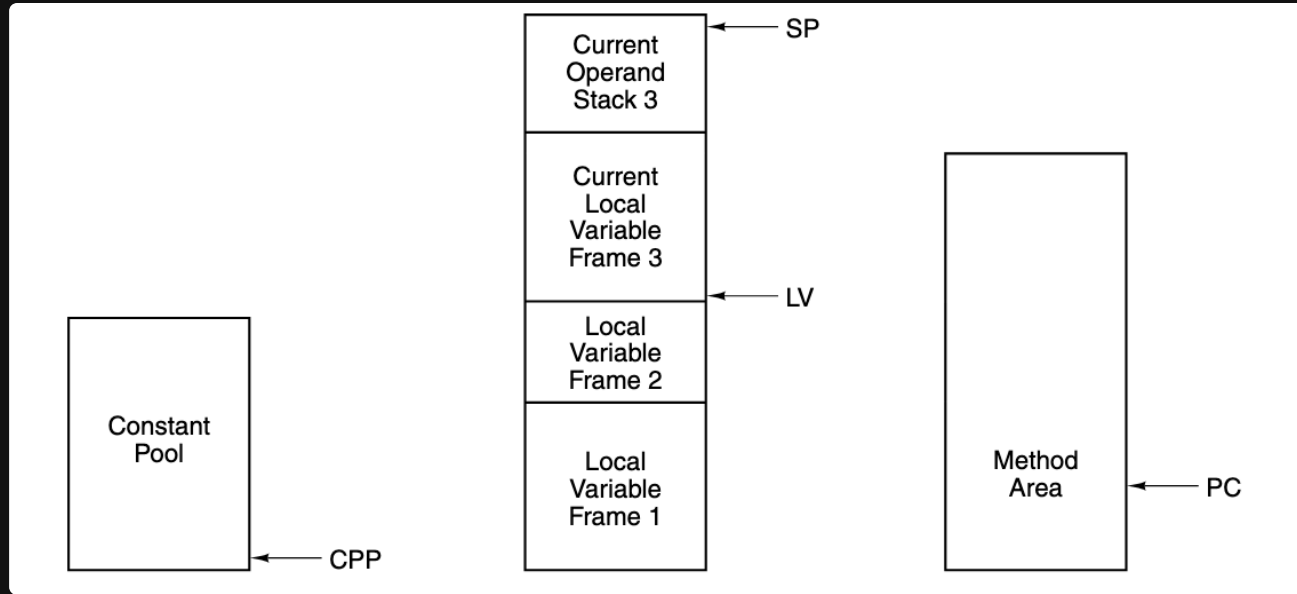
Because JVM programs can only ever refer to memory *relative* to `CPP` , `LV` , `SP` , and `PC` , the same compiled program can run correctly **regardless of where its data actually happens to sit** in physical memory.

This also prevents a program from accidentally (or intentionally) reaching into memory that doesn't belong to it, since there is no way to express an arbitrary absolute address in the first place.

This is in contrast to older, simpler architectures where a compiler might bake in a *fixed absolute address* for a variable

Which, as discussed earlier with recursion, breaks down once multiple simultaneous invocations of the same procedure need separate storage.

The JVM Memory Model



Note that all these pointers are pointers to **words**, not bytes, and are offset by the number of words.

For example, **LV** , **LV + 1** and **LV + 2** point to the first, second and third local variable of the current procedure respectively.

But **PC** (Method Area) is an exception, since it points to **bytes** in the method area, and incrementing it results in a fetch of the next byte.